Vehicle Damage Inspection System

User Manual & Step-by-Step Setup Guide

Automated Pre- and Post-Rental Passenger Car Damage Inspection

MSc Dissertation Prototype — APIIT Sri Lanka

Stefania Crishani · CB016792

Generated 01 September 2026

How to use this manual

This manual assumes **no prior knowledge** of the project, of Python, or of machine learning. Every command you need to type is shown in full, together with what you should see when it works and roughly how long it takes.

Work through it in order. There are two stages:

Stage	Sections	How long	How often
A. Get it running	1 – 5	about 10 minutes	Once per computer
B. Set up the AI model	6 – 9	about 45 minutes, mostly unattended	Once, then only when retraining
Daily use	10 – 12	—	Every time you use the system

You can do Stage A on its own. The application runs immediately without a trained model, using a placeholder that produces fake damage scores so you can explore the screens. Every page and every PDF is clearly marked while this is the case. Stage B replaces the placeholder with a real model trained on 3,937 real damage photographs.

A note on typing commands

Commands are typed into a **terminal** — a window where you type instructions instead of clicking. To open one:

- **macOS**: press Cmd+Space, type *Terminal*, press Enter.
- **Windows**: press the Windows key, type *Command Prompt*, press Enter.

Type each command exactly as shown and press Enter. Lines beginning with # are comments explaining what follows — you do not type those.

1. What this system does

This application replaces the manual walk-around inspection a rental operator performs at vehicle check-out and check-in. Staff photograph the vehicle, a deep-learning model classifies exterior damage, and the system produces a timestamped record. When the vehicle returns, it compares the two inspections and states which damage is **new this rental** and which was **already present at handover**.

The system has two parts, and **both must be running** for it to work:

- **The backend** — a Python program holding the database, the photographs, the damage model and the PDF report generator. You never look at it directly; it runs in a terminal window and answers requests.
- **The frontend** — the dashboard staff actually use, viewed in a web browser.

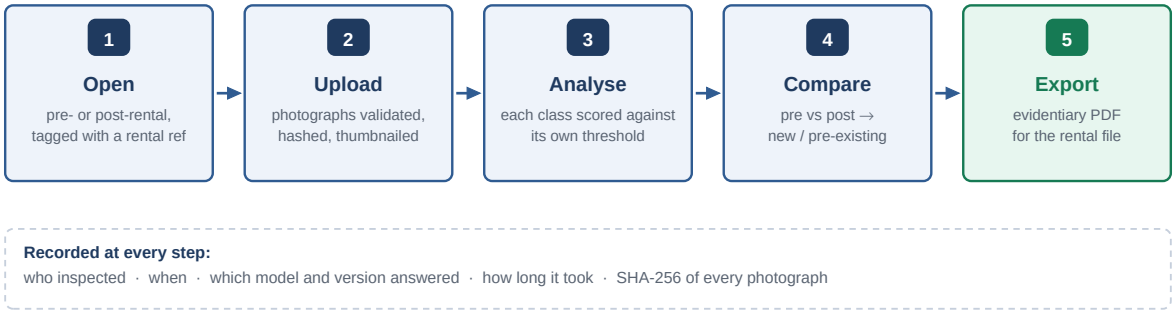

Figure 1. What an inspection looks like end to end. Steps 1 to 3 happen at check-out and again at check-in; step 4 compares the two.

2. Before you start

Two programs must be installed on the computer. Check each one by opening a terminal and typing the command in the third column. If you see a version number, it is installed; if you see *command not found*, install it from the link.

What	Version needed	Check by typing	If missing
Python	3.11 or newer	`python3 --version` (Windows: `python --version`)	python.org/downloads — on Windows you **must** tick *Add python.exe to PATH* during installation
Node.js	18.13+ or 20.9+	`node --version`	nodejs.org — choose the LTS version

You also need:

- **Disk space**: about 1 GB for Stage A, plus 3 GB for Stage B (the dataset and the machine-learning libraries).
- **An internet connection** for the first run of each stage.
- **A modern browser** — Chrome, Edge, Firefox or Safari.

3. Starting the application

You need **two terminal windows open at the same time**, one per part of the system. Both must stay open the whole time you are using the application. Closing a window stops that part.

First, in each window, move into the project folder. Replace the path below with wherever you saved the project:

```
# macOS / Linux
cd ~/projects/car-damage-identification

# Windows
cd C:\Users\YourName\projects\car-damage-identification
```

Step 3.1 — Start the backend (window 1)

```
# macOS / Linux
./run-backend.sh

# Windows
run-backend.bat
```

The first time only, this installs everything the backend needs and takes one to two minutes. Later starts take a few seconds. It has worked when you see this line and the window stops producing new text:

```
INFO:     Uvicorn running on http://127.0.0.1:8000
```

> **Leave this window open.** It looks idle, but it is the running server. If you close it, the dashboard will stop being able to sign in or load anything.

Step 3.2 — Start the frontend (window 2)

Open a **second** terminal window, move into the project folder again, then:

```
# macOS / Linux
./run-frontend.sh

# Windows
run-frontend.bat
```

The first run installs the dashboard's packages and takes a few minutes. It has worked when you see:

```
Local:   http://localhost:4200/
```

Step 3.3 — Open the dashboard

Open your browser and go to **http://localhost:4200**. You should see the sign-in screen. If the page loads but signing in fails, the backend is not running — go back and check window 1.

> **Stopping the system:** press `Ctrl+C` in each window (on Windows, confirm with `Y`), or just close them. Nothing is lost — your records live in `backend/data/app.db` and your photographs in `backend/storage/`, and both are still there next time. The startup scripts also free their ports automatically, so you can simply run them again if you are unsure whether something is already running.

4. Signing in

Two accounts are created automatically the first time the backend starts:

Email	Password	Role	Can do
admin@drivetime.lk	ChangeMe123!	Administrator	Everything, including managing users and editing any inspection
inspector@drivetime.lk	Inspector123!	Inspector	Run inspections and view all records; edit only their own

The sign-in screen lists both accounts — click one to fill the form in automatically, then press **Sign in**. A session lasts 8 hours.

> These are development passwords, printed in this manual and visible in the source code. Change them — and change `SECRET_KEY` in `backend/.env` — before running this anywhere other people can reach.

5. Your first look around

The system starts with five fleet vehicles already registered, so there is something to work with immediately. Take a minute to click through the five items in the left-hand sidebar:

Sidebar item	What it shows
Dashboard	Activity counters, which damage types have been found across the fleet, and the most recent inspections.
Inspections	Every recorded condition check, with filters by type, status and rental reference.
Fleet	The registered vehicles, and each one's inspection history.
Pre / post	The comparison screen — the heart of the system.
Model	Which model is answering and how accurate it has been measured to be. Covered in section 9.

> **Is there an amber banner across the top?** If so it says predictions are placeholders, and that is correct and expected on a fresh installation: no model has been trained yet, so the system generates fake damage scores purely so you can explore the workflow. Until Stage B is done, **do not treat any damage finding as meaningful**.
>
> If there is **no** banner, someone has already completed Stage B for you and the findings are real. You can skip to section 10, though sections 6 to 9 are still worth reading to understand what those findings mean and how they were checked.

Stage B — Setting up the AI model

This is what makes the system actually detect damage rather than pretend to. There are four steps, and you do them **once**:

Step	What it does	Time
6. Install the AI libraries	Adds PyTorch, the machine-learning toolkit	5 min
7. Download the dataset	Fetches 3,937 real damage photographs with expert labels	10 min
7b. Add undamaged cars	Fetches photographs of *clean* cars. Without these the model cannot say 'no damage'	5 min
8. Train the model	Teaches the model to recognise damage. Runs unattended	20–40 min
9. Verify the model	Measures how accurate it actually is	2 min

Every command below is typed in a **new, third terminal window**, and every one starts by moving into the `backend` folder. Leave the two windows from Stage A running.

6. Install the AI libraries

```
cd /path/to/car-damage-identification/backend

# macOS / Linux
.venv/bin/pip install -r requirements-ml.txt

# Windows
.venv\Scripts\pip.exe install -r requirements-ml.txt
```

This downloads roughly 500 MB. When it finishes you will see a line beginning `Successfully installed`.

7. Download the training data

```
# macOS / Linux
.venv/bin/python training/download_dataset.py

# Windows
.venv\Scripts\python.exe training\download_dataset.py
```

This fetches **CarDD** (Wang et al., 2023) — the public research dataset of real car damage photographs, each labelled by human annotators. Progress is printed as it goes, with an estimate of the time remaining.

You should end with:

```
Dataset ready at .../backend/data/CarDD_COCO  (318 MB)
  train: 2816 images
  val  : 810 images
  test : 374 images
  classes: ['dent', 'scratch', 'crack', 'glass shatter', 'lamp broken', 'tire flat']
```

> **The three splits matter and are the basis of section 9.** *train* is what the model learns from. *val* is used during training to check progress. *test* is held back and never shown to the model at all — it is the only honest way to measure accuracy afterwards. If the run is interrupted, simply run the same command again: it downloads only what is missing.

7b. Add undamaged cars (do not skip this)

> **CarDD contains no undamaged vehicles.** All 4,000 photographs show damage. A model trained on that alone is never shown a clean car, so it cannot answer 'no damage' — asked about a pristine vehicle it returns whichever damage class clears its threshold first. Measured, a model trained without this step reported damage on **88.5% of clean cars**. This step is what fixes that.

The photographs come from Stanford Cars, a public dataset of ordinary vehicles. Download one shard and extract it:

```
cd backend
mkdir -p data/negatives_raw
curl -L -o data/negatives_raw/part0.parquet \
  https://huggingface.co/datasets/Multimodal-Fatima/StanfordCars_test/\
resolve/main/data/test-00000-of-00003-18db3ba1d2223f87.parquet

.venv/bin/python training/extract_negatives.py
```

You should end with:

```
Extracted 2681 undamaged car photographs to data/CarDD_COCO/negatives (137 MB)
```

The training and evaluation scripts pick that folder up automatically. They add the clean cars with an all-zero label — no damage of any kind — and split them 70/20/10 across train, validation and test in the same proportions as CarDD itself, so the false-alarm rate is measured on clean cars the model never saw either.

If you skip this step both scripts warn you, and the evaluation refuses to call its headline figure 'damaged versus clean', because without clean cars it is only measuring whether damage is found when damage is present.

8. Train the model

```
# macOS / Linux
.venv/bin/python training/train_classifier.py \
    --data-root data/CarDD_COCO --epochs 30

# Windows (all on one line)
.venv\Scripts\python.exe training\train_classifier.py --data-root data/CarDD_COCO --epochs
 30
```

This is the long step. It runs unattended — start it and do something else. It prints one line per *epoch* (one complete pass over the training images):

```
Training on mps
Trainable parameters: 9,296,100
Epoch 01/20 | train 0.9278 | val 0.6573 | 128s  <- best (saved)
Epoch 02/20 | train 0.6035 | val 0.5510 | 101s  <- best (saved)
Epoch 03/20 | train 0.5232 | val 0.5137 | 104s  <- best (saved)
```

How to read those lines:

- **Training on ...** — `cuda` is an NVIDIA graphics card, `mps` is an Apple Silicon Mac, `cpu` is everything else. CPU works but is several times slower; allow a couple of hours.

- **train** is how well the model does on the images it is learning from. **val** is how well it does on images held aside during training. **val is the one that matters** — it is the honest signal.
- **<- best (saved)** marks an epoch better than every one before it. The checkpoint is written to disk at that moment, so if the run is interrupted you keep the best model so far rather than losing everything.

When train keeps falling but val starts rising

This is **overfitting**: the model has begun memorising the training photographs instead of learning what damage looks like in general. It is normal and expected, and it tells you training has gone as far as it usefully can.

```
Epoch 07/30 | train 0.3793 | val 0.4501 | 75s  <- best (saved)
Epoch 08/30 | train 0.3556 | val 0.4858 | 100s
Epoch 09/30 | train 0.3365 | val 0.4754 | 81s
Epoch 10/30 | train 0.3141 | val 0.4765 | 78s      <- val no longer improving
```

You do not need to watch for this. The script stops on its own once validation has failed to improve for several epochs in a row, and keeps the best epoch:

```
Validation loss has not improved for 5 epochs.
Stopping early at epoch 12; best was epoch 7 (val 0.4501).
```

Control it with `--patience N` (default 8; use `--patience 0` to disable and always run every epoch). Because of this, asking for more epochs than you need costs nothing — the run simply stops when it stops improving.

> **A note for the dissertation.** The original notebook used 60 epochs, which made sense while its backbone was accidentally frozen and only 790,022 parameters had to converge. With that bug fixed, 9,296,100 parameters converge in under ten epochs and then overfit. If you observe the same, it is a genuine result about the corrected setup, not a failed run — and worth reporting as such.

At the end you will see a table of per-class results and:

```
Macro F1 (tuned thresholds): 0.7xxx
Saved checkpoint to models/car_damage_classifier.pth
```

Step 8.1 — Put the model into service

There is a script that performs the whole switch-over — install the checkpoint, restart the API on it, verify it, and rebuild the demo data around it — in one command. This is the recommended route, and it covers steps 8.1, 9 and 10 together:

```
cd backend
./finalize_demo.sh          # macOS / Linux
```

If you would rather do it by hand, or you are on Windows: go to terminal window `1`, press `Ctrl+C` to stop the backend, then start it again with the same command as before. On startup it finds the trained model and loads it automatically — there is nothing to configure.

You know it worked when:

- The startup messages no longer include the warning about placeholder predictions.
- The amber banner has disappeared from every screen in the dashboard.
- The **Model** page shows *Trained weights loaded*.

9. Verifying that the model actually works

Training tells you a model was produced. It does not tell you whether that model is any good. This step measures it, and it is the step to point at when someone asks whether the system really identifies damage.

```
# macOS / Linux
.venv/bin/python training/evaluate.py \
    --data-root data/CarDD_COCO --report

# Windows (all on one line)
.venv\Scripts\python.exe training\evaluate.py --data-root data/CarDD_COCO --report
```

Two things make this a genuine check rather than the model marking its own homework:

1. It scores the **test split only** — 374 photographs the model has never seen, in training or otherwise — against the dataset's own expert labels.
2. It loads the model **through the same code the running application uses** to answer a request. If anything were wrong in the live service, it would be wrong here too.

It prints a table like this, then saves it:

```
class            support  precis  recall      F1      AP     AUC
-------------------------------------------------------------------
dent                 157   0.7xx   0.8xx   0.7xx   0.8xx   0.8xx
scratch              183   0.7xx   0.8xx   0.8xx   0.8xx   0.8xx
...
Exact-match ratio (all six classes right): 0.xxx
Any-damage accuracy (damaged vs clean)   : 0.xxx
```

What the numbers mean

Number	Plain English	Why it matters here
Precision	Of the damage the model reported, how much was really there.	Low precision means **accusing customers of damage that does not exist**. For a dispute tool this is the number that matters most.
Recall	Of the damage that was really there, how much the model found.	Low recall means **missed damage** — the operator absorbs the cost.
F1	A single score balancing the two above.	The headline per-class number. 1.0 is perfect; below about 0.5 the class is unreliable on its own.
Support	How many test images genuinely contain that damage type.	A class with small support has a less trustworthy score, simply because it was measured on fewer examples.
Damaged vs clean	How often it gets 'is this car damaged at all' right, ignoring the type.	The realistic operational number — but **only meaningful if clean cars are in the test set** (step 7b). Without them it measures nothing but recall.
False alarms	How often it reports damage on a car that has none.	The number that decides whether the tool is safe to point at a customer. A high rate means accusing people of damage that is not there.
Exact match	How often all six damage types are simultaneously correct.	A deliberately harsh measure. It will always look low; one wrong class out of six fails the whole image.

Where to see the results

- **In the dashboard**: click **Model** in the sidebar. Headline figures, the per-class table, and a *spot check* list comparing what the model said against the expert label for individual images. Use the **Mistakes only** button to look specifically at what it gets wrong.
- **As a PDF**: `backend/models/evaluation.pdf`, written by the `--report` option. It shows individual photographs beside their labels, **including the failures** — a report showing only successes would prove nothing.
- **As raw data**: `backend/models/evaluation.json`.

9.1 Four ways to satisfy yourself it works

The metrics above are the formal answer. These four checks are the practical ones, easiest first. Run them in front of a supervisor; none needs statistics to read.

Check 1 — Read the measured numbers (30 seconds)

Click **Model** in the sidebar. It shows performance measured on the held-out test split: macro F1, any-damage accuracy, and a per-class table with each class's own threshold and the raw TP / FP / FN counts behind it.

Then press **Mistakes only** in the spot-check list. That filters to the images the model got *wrong*, beside the expert label. A tool that only showed you its successes would not be worth trusting.

Check 2 — Test it against a known answer key (5 minutes)

A ready-made pack is generated for this purpose: fourteen photographs from the held-out split, two per damage type plus two hard multi-damage cases, with a written answer key.

```
cd backend
.venv/bin/python training/make_verify_pack.py

#  ->  data/verify_samples/
#          ANSWER_KEY.md     what each photograph actually shows
#          answer_key.json   the same, machine-readable
#          000033.jpg ...    the photographs
```

1. Open **ANSWER_KEY.md** and keep it beside you.
2. In the dashboard, click **+ New inspection**, pick any vehicle, type a rental reference such as `VERIFY`, and create it.
3. Drag every photograph from `verify_samples/` onto the drop zone and click **Upload and analyse**.
4. Compare what each photograph reports against the answer key.

On the reference run the result was:

```
file          expert says               model says                    verdict
-------------------------------------------------------------------------------
000012.jpg    Tyre flat                 Tyre flat                     EXACT
000015.jpg    Scratch                   Scratch                       EXACT
000090.jpg    Glass shatter             Glass shatter                 EXACT
000297.jpg    Lamp broken               Lamp broken                   EXACT
000033.jpg    Dent                      Dent, Scratch                 found it
000088.jpg    Crack                     Crack, Scratch                found it
000044.jpg    Crack, Dent, Scratch      Crack, Dent, Lamp b., Scratch  found it
-------------------------------------------------------------------------------
Caught the real damage in 14/14.  Exactly right on all six classes: 7/14.
```

Read the *shape* of the near-misses, not just the count. Every one adds an **extra** class rather than missing the real one — 'Dent' becomes 'Dent, Scratch'. For an inspection aid that is the safer direction to err in, because a

human confirms every finding before it reaches a customer. Note also what a second opinion costs: 'scratch' is the class most often added, and it has the lowest threshold of the six.

Check 3 — Reproducibility (2 minutes)

Upload the **same photograph twice** and compare the scores. They must be identical. If the same evidence produced different answers on different days, no report built on it could be defended in a dispute.

```
class              upload 1    upload 2    identical?
-----------------------------------------------------
glass_shatter        1.0000      1.0000    yes
dent                 0.1107      0.1107    yes
scratch              0.0041      0.0041    yes
-----------------------------------------------------
Same SHA-256 fingerprint, identical to four decimal places.
```

Check 4 — The negative control: does it cry wolf?

This is the check that matters most, and the one most easily missed. A system that reports damage on an undamaged car would accuse customers of damage they did not cause.

It is now measured automatically. Provided step 7b was done, the evaluation includes clean cars the model never saw and reports:

```
Damaged vs clean, overall accuracy       : 0.9xx
False alarms on clean cars               : n/269 (x.x%)
```

The **Model** page shows the same figure as a *False alarms* card, coloured red above 15%. If the card is absent, no clean cars were tested and the page says so explicitly rather than implying a pass.

> **Why this check exists.** An early build of this system scored macro F1 0.786 and looked excellent — then reported a dent on a photograph of a pristine Ford Mustang. Measured properly, it was raising false alarms on **88.5% of clean cars**. Nothing in the original metrics could reveal that, because every test image contained damage. The fix was step 7b.

Do the human version too: **photograph an undamaged vehicle yourself and upload it.** A car from your own fleet, in your own lighting, is a harder and more honest test than any curated dataset.

Check	Answers the question	Effort
1. Model page	How accurate is it overall, and where is it weak?	30 s
2. Answer-key pack	Does it get specific, known photographs right?	5 min
3. Reproducibility	Would a report hold up if challenged?	2 min
4. Negative control	Does it cry wolf on an undamaged car? (measured, plus upload your own)	5 min

> **Expect the per-class scores to differ.** Glass shatter and flat tyres are distinctive and score well. Cracks are thin, low-contrast and easily confused with panel gaps or reflections, and score worst. This is a real property of the problem, not a bug — and it is why the dashboard shows each class's own F1 when you hover over a damage label, so a reader can weigh each finding appropriately.

10. Loading the demo with real damage photographs

Optional, but recommended before showing the system to anyone. This fills the database with rentals built from **real photographs from the test split**, pairing a handover image with a return image that carries additional damage, so the comparison screen has genuine new damage to find.

```
# macOS / Linux
.venv/bin/python seed_demo.py --reset

# Windows
.venv\Scripts\python.exe seed_demo.py --reset
```

> `--reset` **deletes every existing inspection** and its photographs before seeding. Vehicles and user accounts are kept. Leave the flag off to add demo rentals alongside whatever is already there.

Each seeded inspection records the dataset's expert labels in its **Notes** field. That turns the dashboard itself into a spot-check: open any inspection and compare the damage the system reports against the ground truth written in the notes.

11. Running an inspection

11.1 Register the vehicle (once per vehicle)

Fleet → **+ Add vehicle**. Registration, make and model are required. The registration is forced to upper case and must be unique — it is how the system identifies the car.

11.2 Record the handover (pre-rental)

1. Click **+ New inspection**, top right of any screen.
2. Choose **Pre-rental**.
3. Select the vehicle.
4. Enter a **rental reference**. **This is the most important field.** It is what links the handover check to the return check. Invent any scheme you like (for example `R-2026-0148`) but use the **identical** reference on both halves of the same rental, or the comparison will not find its pair.
5. Optionally record the odometer, the location and any notes.
6. Click **Create and add photographs**.
7. Choose the **capture angle** from the dropdown, then drag photographs onto the drop zone (or click *browse*). JPEG, PNG or WebP, up to 15 MB each.
8. Click **Upload and analyse**. Results appear when analysis finishes — a second or two per photograph.
9. Change the capture angle and repeat for each side of the car, so the report identifies every photograph correctly.

> Photograph every panel you would check by hand. The system can only report damage that appears in a photograph. An unphotographed panel is **not** evidence of an undamaged panel, and the reports state this explicitly.

11.3 Record the return (post-rental)

Exactly the same, but choose **Post-rental** and enter the **same rental reference** you used at handover.

11.4 Read the result

Each photograph shows all six damage types with a confidence bar. Two things on that bar matter:

- **The coloured bar** is how confident the model is.
- **The small grey tick** is that damage type's decision threshold. Each type has its own, tuned during training — so a raw percentage is not comparable between types, and only a bar that reaches its own tick counts as detected.

Detected types appear in full colour and are listed at the top of the inspection; the rest are dimmed. Hovering over a damage label shows that type's measured F1 score, so you can judge how much weight to give it.

The **severity hint** (minor / moderate / severe) comes from how far a score clears its threshold. It is a display aid only — the model was never taught severity and does not measure how big the damage is.

12. Comparing handover against return

This is what the system is for. Click **Pre / post** in the sidebar, choose the vehicle, and optionally type the rental reference — leave it blank and the most recent pair is used. Click **Compare**.

The verdict panel states the outcome, and the table breaks it down:

Outcome	What it means	Chargeable?
New this rental	Below threshold in every handover photograph, above threshold in at least one return photograph.	**Yes** — it arose during the rental.
Pre-existing	Detected in both inspections.	No — the customer did not cause it.
Not detected on return	Found at handover but not on return. Usually a repair, a different camera angle, or a borderline score either side of the threshold.	No.

Rows marked new are highlighted in red. **Export comparison PDF** produces the document you would attach to a damage claim.

12.1 The two PDF reports

- **Inspection report** — from any inspection, click **Export PDF**. Contains the record, the vehicle, the detected damage with confidences and thresholds, and a page of photographs. Each image carries its SHA-256 fingerprint, its dimensions, which model analysed it and how long that took.
- **Comparison report** — from the Pre / post screen. Both inspections side by side with the damage delta, new damage highlighted.

The SHA-256 fingerprint is recorded when a photograph is uploaded and never recalculated. It lets you prove later that the image behind a finding is the image that was analysed — the property that makes a report usable in a dispute.

13. Troubleshooting

Symptom	Cause and fix
Sign-in fails, or the page says it cannot connect	The backend is not running. Check terminal window 1 and start it again.
`Address already in use`	The startup scripts free their ports automatically, so this is rare. If it happens, find the culprit with `lsof -i :8000` (macOS/Linux) or `netstat -ano | findstr :8000` (Windows) and stop it.
Windows: `'python' is not recognised`	Python is not on your PATH. Reinstall from python.org with *Add python.exe to PATH* ticked, then open a **new** Command Prompt.
Windows: the window closes instantly	Something failed. Run the .bat file from an already-open Command Prompt instead of double-clicking, so the error stays on screen.
Signed out unexpectedly	The 8-hour session expired. Sign in again.
Upload rejected: *not a readable image*	The file is not a JPEG, PNG or WebP, or is damaged. The format is checked by opening the image, not by trusting the file extension.
Upload rejected for size	Over 15 MB. Raise `MAX_UPLOAD_MB` in `backend/.env`, or shrink the photograph.
Amber placeholder banner will not go away	No trained model was found, or it failed to load. Confirm `backend/models/car_damage_classifier.pth` exists, restart the backend, and open the **Model** page.
Training says `no images found`	Step 7 has not been run, or was run in a different folder. Re-run `training/download_dataset.py` from inside `backend`.
Training is extremely slow	It is running on CPU. That is normal and still works — allow a couple of hours, or use `--epochs 10` for a quicker, less accurate model.
Evaluation says the mock predictor is loaded	The trained model is not in place. Complete step 8 and restart the backend before running the evaluation.
It reports damage on a car that is clearly undamaged	Step 7b was skipped, so the model never learned what a clean car looks like and cannot answer 'no damage'. Run `training/extract_negatives.py`, retrain, and check the *False alarms* card on the Model page.
The Model page shows no False alarms card	No clean cars were in the test set, so the rate could not be measured. This is not a pass — it is an unmeasured risk. See step 7b.
The model reports an extra damage type that is not there	Common, and the safer direction to err in. Hover the damage label to see that class's measured F1, and check the confidence bar: a bar only just past its threshold is a weak claim. Every finding is meant to be confirmed by a person.
`make_verify_pack.py` says the dataset is missing	Step 7 has not been run. The pack is drawn from the downloaded test split.
Training stopped before the epoch count you asked for	That is early stopping doing its job: validation had stopped improving. The best epoch was kept. Use `--patience 0` to run every epoch regardless.
Training was interrupted part-way	Nothing is lost. The best epoch so far is already on disk as `models/car_damage_classifier.best.pth`; `finalize_demo.sh` will promote it, or copy it over `car_damage_classifier.pth` yourself.
Downloads hang or time out	Some networks block Python's own downloader. Both the dataset script and the training script fall back to `curl` automatically, so re-running the command usually succeeds.
Want to wipe everything and start again	Stop the backend, delete `backend/data/app.db` and the `backend/storage/` folder, then start it again. Accounts and vehicles are recreated; inspections are not.

14. Quick reference

Every command, in order

```
# ---- Stage A: run the application (two terminal windows) ----
./run-backend.sh          # window 1   (Windows: run-backend.bat)
./run-frontend.sh         # window 2   (Windows: run-frontend.bat)
#    then open http://localhost:4200

# ---- Stage B: set up the model (third window, inside backend/) ----
cd backend
.venv/bin/pip install -r requirements-ml.txt        # 6. libraries
.venv/bin/python training/download_dataset.py       # 7. damaged cars
#    7b. undamaged cars - see section 7b for the curl command
.venv/bin/python training/extract_negatives.py      # 7b. clean cars
.venv/bin/python training/train_classifier.py \
    --data-root data/CarDD_COCO \
    --epochs 20 --patience 5                        # 8. train

./finalize_demo.sh          # 8.1 + 9 + 10 in one step:
                            #   installs the checkpoint, restarts the
                            #   API on it, verifies it against held-out
                            #   data, re-seeds the demo

# ---- or do those three by hand ----
#   restart the backend so it picks up the model
.venv/bin/python training/evaluate.py \
    --data-root data/CarDD_COCO --report            # 9. verify
.venv/bin/python seed_demo.py --reset               # 10. demo data

# ---- checking it by hand (section 9.1) ----
.venv/bin/python training/make_verify_pack.py       # answer-key pack
#    then drag data/verify_samples/*.jpg into a new inspection

# On Windows replace  .venv/bin/python  with  .venv\Scripts\python.exe
#                and  .venv/bin/pip     with  .venv\Scripts\pip.exe
```

Addresses

Address	What it is
`http://localhost:4200`	The dashboard — this is the one you use
`http://localhost:8000/docs`	Interactive API documentation, for testing endpoints directly
`http://localhost:8000/api/v1/model`	Which model is loaded right now. `"is_real": true` means a trained model

Where things are kept

Path	Contents
`backend/data/app.db`	All records: users, vehicles, inspections, findings
`backend/data/CarDD_COCO/`	The training dataset (only after step 7)
`backend/data/CarDD_COCO/negatives/`	Undamaged cars, added in step 7b
`backend/data/verify_samples/`	The hand-verification pack and its answer key (section 9.1)
`backend/storage/inspections/`	Uploaded photographs and thumbnails
`backend/models/`	The trained model (`car_damage_classifier.pth`), the best-so-far checkpoint written during training (`...best.pth`), and the evaluation report (`evaluation.json` / `.pdf`)
`backend/finalize_demo.sh`	One-shot: install the model, restart the API, verify, re-seed
`backend/.env`	Settings — passwords, limits, ports
`docs/`	This manual and the technology document